

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Escuela de Ingenieria de Software

GUIA DE LABORATORIO 05

CURSO DE TALLER DE CONSTRUCCION DE SOFTWARE MOVIL

**CLASIFICACION DE TUMORES CEREBRALES EN IMAGENES
MRI**

MEDIANTE REACT NATIVE, FLASK Y CNN

DR. IVAN CARLO PETRLIK AZABACHE

LIMA - PERU

2026

PARTE 1: CONFIGURACION DEL BACKEND (FLASK)

Esta seccion es identica a la guia original. El backend Flask sirve el modelo CNN de TensorFlow Lite para clasificar imagenes MRI de tumores cerebrales. No requiere cambios al migrar de Flutter a React Native.

1.1 Crear entorno virtual e instalar dependencias

Abrir una terminal (PowerShell en Windows) y ejecutar los siguientes comandos:

```
py -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.venv\Scripts\activate

python.exe -m pip install --upgrade pip
pip install flask
pip install gunicorn
pip install flask-cors
python -m pip install tensorflow-cpu
pip install numpy matplotlib pandas
pip freeze > requirements.txt
```

1.2 Descargar el dataset de Kaggle

Descargar el dataset de imagenes MRI desde la plataforma Kaggle, el cual sera utilizado para el entrenamiento y validacion del modelo de inteligencia artificial.

Enlace del dataset:

<https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset?resource=download>

Subir las carpetas Training y Testing al Google Drive dentro de una carpeta llamada datasetMRI.

1.3 Entrenar el modelo en Google Colab

Acceder al siguiente enlace de Google Colab para ejecutar el codigo de entrenamiento del modelo CNN:

<https://colab.research.google.com/drive/1sScbGhKmARWuZNXDf77NPptH1aXMoRtZ?usp=sharing>

El codigo completo realiza las siguientes operaciones:

1. Monta Google Drive y carga el dataset
2. Crea generadores de imagenes con rescaling (1/255)
3. Construye un modelo CNN con 3 capas convolucionales + MaxPooling + Dropout
4. Compila con Adam optimizer y categorical_crossentropy
5. Entrena durante 10 epocas

6. Convierte el modelo a formato TensorFlow Lite (.tflite)
7. Genera graficos de precision/perdida y reporte de clasificacion

1.4 Descargar el modelo TFLite

Descargar el modelo de IA generado en formato TFLite desde el siguiente enlace:

https://drive.google.com/file/d/1ocZQoUYXR01sglJVpEf9XlsG0Bg3j_-C/view?usp=sharing

Colocar el archivo brain_tumor_cnn.tflite en la carpeta raiz del proyecto Flask.

1.5 Crear el archivo backend.py

Crear el archivo backend.py en la raiz del proyecto con el siguiente codigo:

backend.py

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import tensorflow as tf
import numpy as np
from tensorflow.keras.preprocessing import image
import os
from werkzeug.utils import secure_filename
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

app = Flask(__name__)
CORS(app)

interpreter = tf.lite.Interpreter(
    model_path='brain_tumor_cnn.tflite'
)
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

class_names = ['glioma', 'meningioma', 'notumor', 'pituitary']

UPLOAD_FOLDER = os.path.join('static', 'uploads')
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

def predict_with_tflite(img_array):
    img_array = img_array.astype(np.float32) / 255.0
    interpreter.set_tensor(
        input_details[0]['index'], img_array)
    interpreter.invoke()
    output_data = interpreter.get_tensor(
        output_details[0]['index'])
    return output_data[0]

@app.route('/api/clasificar', methods=['POST'])
def clasificar_api():
    file = request.files.get('image')
    if not file:
        return jsonify({'error': 'No se envio imagen'}), 400
```

```
filename = secure_filename(file.filename)
filepath = os.path.join(
    app.config['UPLOAD_FOLDER'], filename)
file.save(filepath)

img = image.load_img(
    filepath, target_size=(128, 128))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)

prediction = predict_with_tflite(img_array)
predicted_class = class_names[np.argmax(prediction)]
probabilities = {
    class_names[i]: float(f'{prob:.4f}')
    for i, prob in enumerate(prediction)
}

return jsonify({
    'prediction': f'Prediccion: {predicted_class.upper()}',
    'image_name': filename,
    'probs': probabilities
})

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

1.6 Ejecutar el backend

Ejecutar el servidor Flask:

```
(.venv) PS C:\MODELO_PREDICTIVO_DTC> python backend.py
```

El servidor estara disponible en <http://localhost:5000>

ARQUITECTURA DEL SISTEMA

La arquitectura mantiene la misma estructura cliente-servidor de la guia original, reemplazando Flutter por React Native como tecnologia del frontend movil:

Componente	Tecnologia	Responsabilidad
Frontend Movil	React Native (Expo)	Interfaz de usuario, captura de imagen, visualizacion de resultados
Backend API	Flask (Python)	Recibe imagen, ejecuta inferencia con TFLite, retorna prediccion
Modelo IA	TensorFlow Lite	Modelo CNN entrenado para clasificar 4 tipos de tumores cerebrales
Comunicacion	HTTP REST (multipart/form-data)	POST /api/clasificar con imagen adjunta

PARTE 2: DESARROLLO DE LA APLICACION MOVIL (REACT NATIVE)

A continuacion, se desarrollara la aplicacion movil en React Native utilizando Expo, reemplazando la implementacion original en Flutter.

2.1 Crear el proyecto React Native con Expo

Abrir una terminal y ejecutar los siguientes comandos para crear el proyecto:

```
npx create-expo-app@latest BrainTumorApp --template blank
cd BrainTumorApp
```

Instalar las dependencias necesarias:

```
npx expo install expo-image-picker
npm install axios
```

2.2 Estructura del proyecto (Patron MVC)

Dentro de la carpeta del proyecto, crear la siguiente estructura de archivos y carpetas:

```
BrainTumorApp/
  src/
    models/
      ClasificacionResult.js
    services/
      apiService.js
    controllers/
      useClasificacion.js
    views/
      ClasificacionView.js
    components/
      ResultCard.js
      ProbabilityChart.js
  App.js
```

Capa	Archivo	Responsabilidad
Model	models/ClasificacionResult.js	Mapea el JSON del backend Flask
Service	services/apiService.js	POST multipart al endpoint /api/clasificar
Controller	controllers/useClasificacion.js	Hook con estado reactivo, orquesta picker y API
View	views/ClasificacionView.js	UI completa, sin logica de negocio
Components	components/ResultCard.js	Tarjeta de resultado con descripcion clinica
Components	components/ProbabilityChart.js	Grafico de barras de probabilidades

2.3 Código fuente completo

Model: ClasificacionResult.js

src/models/ClasificacionResult.js

```
// src/models/ClasificacionResult.js

export default class ClasificacionResult {
  constructor(data) {
    this.prediction = data.prediction || '';
    this.imageName = data.image_name || '';
    this.probs = data.probs || {};
  }

  get predictedClass() {
    return this.prediction
      .replace('Prediccion: ', '')
      .trim();
  }

  get probsArray() {
    return Object.entries(this.probs).map(
      ([name, value]) => ({
        name: name.charAt(0).toUpperCase()
          + name.slice(1),
        value: parseFloat(value),
        percentage: (parseFloat(value) * 100)
          .toFixed(1)
      })
    );
  }

  getColorForClass(className) {
    const colors = {
      GLIOMA: '#E74C3C',
      MENINGIOMA: '#F39C12',
      NOTUMOR: '#27AE60',
      PITUITARY: '#3498DB',
    };
    return colors[className] || '#95A5A6';
  }

  get description() {
    const desc = {
      GLIOMA: 'Tumor originado en células gliales.',
      MENINGIOMA: 'Tumor en las meninges.',
      NOTUMOR: 'No se detecta tumor.',
      PITUITARY: 'Tumor en la glándula pituitaria.',
    };
    return desc[this.predictedClass]
      || 'Clasificación desconocida.';
  }
}
```

Service: apiService.js

src/services/apiService.js

```
// src/services/apiService.js
import axios from 'axios';

// Cambiar por la IP de tu maquina en la red local
const BASE_URL = 'http://192.168.1.100:5000';

export async function clasificarImagen(imageUri) {
  const formData = new FormData();

  // Extraer nombre y extension del URI
  const filename = imageUri.split('/').pop();
  const ext = filename.split('.').pop();
  const mimeType = ext === 'png'
    ? 'image/png'
    : 'image/jpeg';

  formData.append('image', {
    uri: imageUri,
    name: filename,
    type: mimeType,
  });

  try {
    const response = await axios.post(
      `${BASE_URL}/api/clasificar`,
      formData,
      {
        headers: {
          'Content-Type': 'multipart/form-data',
        },
        timeout: 30000,
      }
    );
    return response.data;
  } catch (error) {
    if (error.response) {
      throw new Error(
        `Error del servidor: ${error.response.status}`
      );
    } else if (error.request) {
      throw new Error(
        'No se pudo conectar con el servidor'
      );
    }
    throw error;
  }
}
```

Controller: useClasificacion.js

src/controllers/useClasificacion.js

```
// src/controllers/useClasificacion.js
import { useState } from 'react';
import * as ImagePicker from 'expo-image-picker';
import { clasificarImagen } from
  '../services/apiService';
import ClasificacionResult from
```

```
'../models/ClasificacionResult';

export default function useClasificacion() {
  const [imageUri, setImageUri] = useState(null);
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const pickImage = async () => {
    const permResult = await ImagePicker
      .requestMediaLibraryPermissionsAsync();

    if (!permResult.granted) {
      setError('Se requiere permiso de galeria');
      return;
    }

    const pickerResult = await ImagePicker
      .launchImageLibraryAsync({
        mediaTypes: ImagePicker.MediaTypeOptions.Images,
        quality: 0.8,
      });

    if (!pickerResult.canceled) {
      const uri = pickerResult.assets[0].uri;
      setImageUri(uri);
      setResult(null);
      setError(null);
      await clasificar(uri);
    }
  };

  const clasificar = async (uri) => {
    setLoading(true);
    setError(null);
    try {
      const data = await clasificarImagen(uri);
      setResult(
        new ClasificacionResult(data)
      );
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  const reset = () => {
    setImageUri(null);
    setResult(null);
    setError(null);
  };

  return {
    imageUri, result, loading,
    error, pickImage, reset
  };
}
```

Component: ProbabilityChart.js

src/components/ProbabilityChart.js

```
// src/components/ProbabilityChart.js
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function ProbabilityChart({ probs }) {
  const colors = [
    '#E74C3C', '#F39C12',
    '#27AE60', '#3498DB'
  ];

  return (
    <View style={styles.container}>
      <Text style={styles.title}>
        Probabilidades por Clase
      </Text>
      {probs.map((item, index) => (
        <View key={item.name}
          style={styles.barRow}>
          <Text style={styles.label}>
            {item.name}
          </Text>
          <View style={styles.barBg}>
            <View style={[
              styles.barFill,
              {
                width: `${item.value * 100}%`,
                backgroundColor:
                  colors[index % colors.length],
              },
            ]}>
            </View>
          </View>
          <Text style={styles.percent}>
            {item.percentage}%
          </Text>
        </View>
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    backgroundColor: '#fff',
    borderRadius: 12, padding: 16,
    marginTop: 16,
    elevation: 2,
  },
  title: {
    fontSize: 16, fontWeight: 'bold',
    marginBottom: 12, color: '#333',
  },
  barRow: {
    flexDirection: 'row',
    alignItems: 'center',
    marginBottom: 8,
  },
  label: {
    width: 90, fontSize: 13,
    color: '#555',
  },
});
```

```

    },
    barBg: {
      flex: 1, height: 20,
      backgroundColor: '#EEE',
      borderRadius: 10,
      overflow: 'hidden',
    },
    barFill: {
      height: '100%', borderRadius: 10,
    },
    percent: {
      width: 50, textAlign: 'right',
      fontSize: 13, fontWeight: '600',
      color: '#333',
    },
  },
});

```

Component: ResultCard.js

src/components/ResultCard.js

```

// src/components/ResultCard.js
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function ResultCard({ result }) {
  const color = result.getColorForClass(
    result.predictedClass
  );

  return (
    <View style={[
      styles.card,
      { borderLeftColor: color }
    ]>
      <Text style={[
        styles.prediction,
        { color }
      ]>
        {result.prediction}
      </Text>
      <Text style={styles.description}>
        {result.description}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: '#fff',
    borderRadius: 12,
    padding: 16,
    marginTop: 16,
    borderLeftWidth: 5,
    elevation: 2,
  },
  prediction: {
    fontSize: 20,
    fontWeight: 'bold',
    marginBottom: 8,
  },
  description: {

```

```
fontSize: 14,  
color: '#666',  
lineHeight: 20,  
},  
});
```

View: ClasificacionView.js

src/views/ClasificacionView.js

```
// src/views/ClasificacionView.js
import React from 'react';
import {
  View, Text, Image,
  TouchableOpacity, ScrollView,
  ActivityIndicator, StyleSheet,
  SafeAreaView,
} from 'react-native';
import useClasificacion from
  '../controllers/useClasificacion';
import ResultCard from
  '../components/ResultCard';
import ProbabilityChart from
  '../components/ProbabilityChart';

export default function ClasificacionView() {
  const {
    imageUrl, result, loading,
    error, pickImage, reset
  } = useClasificacion();

  return (
    <SafeAreaView style={styles.safe}>
      <ScrollView
        contentContainerStyle={styles.scroll}>

        {/* Header */}
        <View style={styles.header}>
          <Text style={styles.title}>
            Clasificador de Tumores
          </Text>
          <Text style={styles.subtitle}>
            Cerebrales (MRI)
          </Text>
        </View>

        {/* Image Preview */}
        <View style={styles.imageContainer}>
          {imageUrl ? (
            <Image
              source={{ uri: imageUrl }}
              style={styles.image}
              resizeMode='cover'
            />
          ) : (
            <View style={styles.placeholder}>
              <Text style={styles.placeholderText}>
                Seleccione una imagen MRI
              </Text>
            </View>
          )}
        </View>

        {/* Buttons */}
        <TouchableOpacity
          style={styles.button}
          onPress={pickImage}
          disabled={loading}>
          <Text style={styles.buttonText}>
```

```

        {loading
          ? 'Clasificando...'
          : 'Seleccionar Imagen'}
      </Text>
    </TouchableOpacity>

    {result && (
      <TouchableOpacity
        style={styles.resetBtn}
        onPress={reset}>
        <Text style={styles.resetText}>
          Nueva Clasificacion
        </Text>
      </TouchableOpacity>
    )}

    {/* Loading */}
    {loading && (
      <ActivityIndicator
        size='large'
        color='#00B4D8'
        style={{ marginTop: 20 }}
      />
    )}

    {/* Error */}
    {error && (
      <View style={styles.errorBox}>
        <Text style={styles.errorText}>
          {error}
        </Text>
      </View>
    )}

    {/* Results */}
    {result && (
      <>
        <ResultCard result={result} />
        <ProbabilityChart
          probs={result.probsArray}
        />
      </>
    )}
  </ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: '#F0F4F8' },
  scroll: { padding: 20 },
  header: { alignItems: 'center', marginBottom: 20 },
  title: {
    fontSize: 24, fontWeight: 'bold',
    color: '#1B3A5C',
  },
  subtitle: {
    fontSize: 16, color: '#666',
    marginTop: 4,
  },
  imageContainer: {
    alignItems: 'center', marginBottom: 16,
  },
});

```

```
image: {
  width: 280, height: 280,
  borderRadius: 16,
},
placeholder: {
  width: 280, height: 280,
  borderRadius: 16,
  backgroundColor: '#DDD',
  justifyContent: 'center',
  alignItems: 'center',
},
placeholderText: { color: '#888', fontSize: 14 },
button: {
  backgroundColor: '#00B4D8',
  paddingVertical: 14,
  borderRadius: 12,
  alignItems: 'center',
},
buttonText: {
  color: '#fff', fontSize: 16,
  fontWeight: 'bold',
},
resetBtn: {
  marginTop: 10, paddingVertical: 12,
  borderRadius: 12,
  borderWidth: 1,
  borderColor: '#00B4D8',
  alignItems: 'center',
},
resetText: { color: '#00B4D8', fontSize: 14 },
errorBox: {
  backgroundColor: '#FDECEA',
  padding: 12, borderRadius: 8,
  marginTop: 16,
},
errorText: { color: '#C0392B', fontSize: 14 },
});
```

App.js (punto de entrada)

App.js

```
// App.js
import React from 'react';
import ClasificacionView from
  './src/views/ClasificacionView';

export default function App() {
  return <ClasificacionView />;
}
```

2.4 Configuración de red (importante)

Para que la app React Native se conecte al backend Flask, es necesario configurar la IP correcta:

1. Obtener la IP local de tu máquina ejecutando **ipconfig** (Windows) o **ifconfig** (Mac/Linux)
2. Editar **src/services/apiService.js** y reemplazar la IP en **BASE_URL**
3. Asegurarse de que el celular y la computadora estén en la misma red WiFi
4. Ejecutar Flask con **app.run(host='0.0.0.0', debug=True, port=5000)** para aceptar conexiones externas

2.5 Ejecutar la aplicación

Con el backend Flask corriendo, ejecutar la app React Native:

```
cd BrainTumorApp
npx expo start
```

Escanear el código QR con la app Expo Go en el celular, o presionar 'a' para abrir en emulador Android.

COMPARATIVA: FLUTTER vs REACT NATIVE

La siguiente tabla resume las diferencias principales entre la implementación original en Flutter y la versión adaptada a React Native:

Aspecto	Flutter (Original)	React Native (Adaptado)
Lenguaje	Dart	JavaScript (ES6+)
Framework CLI	flutter create	npx create-expo-app
Gestor de paquetes	pubspec.yaml (pub)	package.json (npm)
HTTP Client	http / dio	axios
Image Picker	image_picker	expo-image-picker
Graficos	fl_chart	Componente custom con View
Patrón MVC	Clases + StatefulWidget	Hooks + Functional Components
Estado	setState() en StatefulWidget	useState() Hook
Styling	Widget properties	StyleSheet.create()
Build APK	flutter build apk	eas build -p android
Hot Reload	Si	Si (Fast Refresh)

EJERCICIO PROPUESTO

En base al ejercicio desarrollado, se deberá:

5. Desplegar la aplicación del proyecto Flask en el servicio Render (<https://render.com>).
6. Actualizar la URL del backend en apiService.js para apuntar al servicio desplegado en Render.
7. Compilar la aplicación React Native y generar su respectivo archivo APK para instalación y prueba en dispositivos Android, utilizando el comando: eas build -p android --profile preview